POS TECH

TECH CHALLENGE

# FASE 02

# TECH CHALLENGE

Tech Challenge é o projeto que engloba os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, em princípio, deve ser desenvolvida em grupo. Importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale **90%** da nota de todas as disciplinas da fase.

**Nota Importante sobre o Ambiente de Nuvem**

Este desafio foi projetado para ser flexível. Você tem 2 (duas) opções principais para seu ambiente de nuvem:

**Opção A:** AWS Academy (Ambiente com Limitações)

1. Se você está usando sua conta do AWS Academy, você **não pode criar novas roles de IAM**.
2. Você **deve** usar a role existente chamada **LabRole** para todas as operações que exigem permissão (criação do cluster, node groups, etc.).
3. Ferramentas modernas como KEDA e Karpenter **não funcionarão**, pois dependem da criação de novas roles (IRSA).
4. **Este documento fornecerá o caminho e os workarounds necessários para ser bem-sucedido neste ambiente.**

**Opção B**: Conta Pessoal (AWS, GCP, Azure, OCI)

1. Este é o caminho profissional recomendado se você tiver a possibilidade. Você pode usar uma conta AWS pessoal (com acesso total ao IAM), GCP (Google Kubernetes Engine), Azure (Azure Kubernetes Service) ou qualquer outra.
2. Você poderá usar as ferramentas padrão do mercado (eksctl, helm, IRSA, KEDA) sem limitações.

3 - O desafio será o mesmo, mas você poderá usar métodos mais modernos (e recomendados) para escalabilidade.

As instruções a seguir tentarão cobrir ambas as opções, destacando quando os caminhos divergem.

## Desafio

Parabéns! O MVP monolítico do ToggleMaster implantado na Fase 1 foi um sucesso. A plataforma validou sua utilidade e agora a demanda explodiu. A arquitetura monolítica começou a apresentar gargalos, e a diretoria da DevOps Solutions Inc. decidiu que é hora de evoluir: o ToggleMaster será reescrito como um ecossistema de microsserviços distribuídos.

O seu desafio nesta fase é pegar o código-fonte dos 5 novos microsserviços, conteinerizá-los, provisionar a infraestrutura de nuvem necessária e implantá-los em um ambiente de orquestração robusto, escalável e resiliente: o Kubernetes na AWS (EKS).

A arquitetura foi quebrada nos seguintes 5 microsserviços (você receberá o código-fonte de todos eles):

- **auth-service (Go):** Gerencia chaves de API e autenticação. (Banco de Dados: PostgreSQL)
- **flag-service (Python):** CRUD das definições das feature flags. (Banco de Dados: PostgreSQL)
- **targeting-service (Python):** Gerencia regras complexas de segmentação. (Banco de Dados: PostgreSQL)
- **evaluation-service (Go):** O "caminho quente" (hot path) de alta performance que retorna a decisão final (true/false). (Cache: Redis)
- **analytics-service (Python):** Consome eventos de uma fila e salva dados de análise. (Fila: AWS SQS, Banco de Dados: AWS DynamoDB)

Sua missão é projetar e implementar a infraestrutura de contêineres e orquestração para colocar esse novo ecossistema em produção.

**Código usado durante o Tech Challenge**

O código para utilizar no desafio pode ser pego aqui:

## REQUISITOS TÉCNICOS

### 1 - Análise e Conteinerização (Docker)

O primeiro passo é garantir que você consiga executar e entender todo o ecossistema localmente.

- **Dockerfile:** Você deve criar um Dockerfile otimizado (multi-stage builds são altamente recomendados) para cada um dos 5 microsserviços.
- **Docker Compose:** Crie um único arquivo docker-compose.yml na raiz do projeto que suba todos os 5 microsserviços e seus 4 bancos de dados locais (2 instâncias PostgreSQL, 1 Redis, 1 DynamoDB Local). Isso é essencial para provar que o ambiente local funciona.

### 2 - Provisionando a Infraestrutura na Nuvem (Console AWS e eksctl)

Antes de implantar, você deve provisionar manualmente (via Console da AWS ou eksctl) todos os recursos de nuvem que seus microsserviços precisarão.

Este é o seu **checklist de infraestrutura**:

**Cluster Kubernetes**

**Opção A (AWS Academy):**

- ✓ Crie 1 (um) cluster **AWS EKS** usando o Console da AWS. **Não** use o eksctl create cluster.
- ✓ Cluster Role**:** Quando solicitado, selecione a role existente LabRole.
- ✓ Crie um **Managed Node Group** (pelo console).

- ✓ Node IAM Role**:** Quando solicitado, selecione a **LabRole** existente. (Isso é crucial! Os nós herdarão as permissões desta role).
- ✓ Configuração de Auto Scaling**:** Defina a configuração de escalabilidade do grupo de nós (ex: Mínimo=1, Desejado=2, Máximo=4 instâncias).

**Opção B (Conta Pessoal - AWS, GCP, Azure, OCI...):**

Você tem liberdade. Crie um cluster Kubernetes.

- ✓ **Se usar AWS:** O método recomendado é usar eksctl create cluster. Ele criará automaticamente as roles de IAM corretas (não precisará da LabRole).
- ✓ **Se usar GCP/Azure:** Use o GKE ou AKS, que simplificam muito essa etapa.

**Registro de Contêineres (ECR):**

- ✓ Crie 5 (cinco) repositórios no **AWS ECR**, um para cada microsserviço (ex: auth-service, flag-service, etc.).
- ✓ Publique as imagens Docker que você criou na etapa 1 para seus respectivos repositórios no ECR.

**Bancos de Dados Relacionais (RDS):**

Crie 3 (três) instâncias de banco de dados **AWS RDS for PostgreSQL** independentes.

- ✓ Recurso 1 (RDS): Para o auth-service.
- ✓ Recurso 2 (RDS): Para o flag-service.
- ✓ Recurso 3 (RDS): Para o targeting-service.

**Cache In-Memory (ElastiCache):**

- ✓ Crie 1 (um) cluster **AWS ElastiCache for Redis**.

- ✓ Recurso 4 (ElastiCache): Para o evaluation-service.

**Banco de Dados NoSQL (DynamoDB):**

- ✓ Crie 1 (uma) tabela no **AWS DynamoDB**.
- ✓ Recurso 5 (DynamoDB): Para o analytics-service. (O código-fonte indicará o nome da tabela e a chave primária esperada).

**Fila de Mensagens (SQS):**

- ✓ Crie 1 (uma) fila **AWS SQS** (do tipo Standard).
- ✓ Recurso 6 (SQS): Para ser usada pelo evaluation-service (que produz mensagens) e pelo analytics-service (que consome as mensagens).

| |
|---|
| Ao final desta etapa, você deve ter anotado todas as "strings de conexão" (endpoints do RDS, endpoint do ElastiCache, nome da tabela DynamoDB, ARN da fila SQS, etc.), pois você as usará na próxima etapa. |

## 3 - Configurando o Cluster (Kubernetes)

**Metrics Server - *Comum a ambas as opções***

- ✓ Instale o Metrics Server no seu cluster. Ele é necessário para o HPA funcionar. (Você pode usar kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml).

**Nginx Ingress Controller**

- ✓ **Opção A (AWS Academy):** Instale o Nginx Ingress Controller (via Helm ou kubectl apply). Como seus nós têm a LabRole, o Nginx Controller terá permissão para criar um Application Load Balancer (ALB) ou Network Load Balancer (NLB) na AWS.

- ✓ **Opção B (Conta Pessoal):** Instale o Nginx Ingress Controller. O método recomendado é usar **IRSA** (IAM Roles for Service Accounts) para dar ao pod do controller permissão para gerenciar Load Balancers, sem dar permissão total ao nó.

**4 - Orquestração e Implantação (Manifestos) - Comum a ambas as opções**

Agora, escreva os manifestos do Kubernetes para implantar suas aplicações.

**Manifestos Básicos:** Crie os arquivos YAML para cada um dos 5 microsserviços:

1. **Namespaces** (separadores lógicos para aplicações).
2. **Deployment** (para gerenciar os Pods, garantindo que eles usem as imagens do ECR).
3. **Service** (do tipo ClusterIP).
4. **Secrets** (para injetar com segurança todas as senhas, endpoints e chaves de acesso dos recursos que você criou na Etapa 2).
5. **ConfigMap** (para injetar URLs de serviços internos e outros dados).

**Acesso Externo (Ingress):**

Crie um manifesto Ingress que defina as regras de roteamento (ex: /auth vai para o auth-service, /flags para o flag-service, etc.).

**Boas práticas de orquestração:**

- ✓ Use sempre Requests e Limits nos Deployments para evitar problemas com o Node.
- ✓ Garanta que as secrets sempre estarão em base64.
- ✓ Use sempre Readiness e/ou LivenessProbe sempre que possível

- ✓ Crie sempre suas aplicações separando por Namespaces.

## 5 - Configurando a Escalabilidade

### Horizontal Pod Autoscaler (HPA) - Requisito Mínimo (Opção A)

- ✓ Crie um manifesto HorizontalPodAutoscaler para o **evaluation-service** baseado na **utilização média de CPU** (ex: targetCPUUtilizationPercentage: 70).
- ✓ Crie um manifesto HorizontalPodAutoscaler para o **analytics-service** também baseado na **utilização média de CPU**.
- ✓ Explicação (Workaround): Esta é a solução para o Academy. Quando a fila SQS encher, este serviço processará mais mensagens, sua CPU aumentará, e o HPA adicionará mais pods.

### (Opcional) KEDA - Recomendado (Opção B)

- ✓ Se você estiver em uma conta pessoal, em vez de usar HPA por CPU para o analytics-service, o desafio real é usar o **KEDA** (Kubernetes Event-driven Autoscaling).
- ✓ Crie um ScaledObject para o analytics-service que monitore diretamente a fila SQS (queueDepth) e escale os pods de 0 para N.

## ENTREGÁVEIS DA FASE 2

**Vídeo de Demonstração (até 20 min):**

- Mostre o docker compose up funcionando, com todos os 9 contêineres (5 apps + 4 DBs) rodando localmente.
- Mostre seu cluster Kubernetes provisionado na nuvem (EKS, GKE, ou AKS).
- Mostre os 5 microsserviços rodando como Pods (kubectl get pods).
- Mostre o Nginx Ingress funcionando (faça uma chamada curl ou Postman para a URL do Load Balancer criado).

**Demonstre a escalabilidade:**

- Gere carga no evaluation-service (com hey, ab ou Postman) e mostre o HPA aumentando as réplicas (kubectl get hpa e kubectl get pods).
- Envie manualmente várias mensagens para a fila SQS.
- Mostre o HPA (ou KEDA, se usou) do analytics-service detectando a carga (CPU ou fila) e escalando seus pods.
- Mostre os dados aparecendo na tabela do DynamoDB.
- Faça uma breve explicação da arquitetura e dos desafios encontrados (ex: limitações da LabRole).
- Explique como você implementou a escalabilidade do analytics-service (HPA por CPU ou KEDA por fila) e justifique a escolha.
- Explique a diferença de propósito entre os 3 *data stores* utilizados (RDS, ElastiCache e DynamoDB).

**Relatório de Entrega (.PDF ou .txt):**

- Nomes dos participantes, RM e usernames do Discord.
- Link dos repositórios.

- Link do vídeo salvo no Youtube ou lugar de sua preferência.

**Pontuação extra (*A pontuação máxima por fase é de 90 pontos*):**

Vocês receberão um convite para o Google Cloud Skills Boost com uma trilha de curso para aprimorar seus conhecimentos.

A conclusão da trilha é totalmente opcional e não sofrerá na perda de pontos caso não seja finalizada.

Mas, para aqueles que concluírem a trilha e deixar o perfil/badge pública e adicionar o link no Relatório de Entrega, ganhará 10 pontos extras nessa Fase 2 do Tech Challenge.

Lembrando que qualquer dúvida, podem nos chamar no Discord.

POS TECH